

Algoritmos Mínimos: Kruskal vs. Prim

Guía de referencia rápida y criterios de selección para el Árbol Cobertor Mínimo (MST)

Esta guía ofrece un análisis detallado sobre cómo funcionan y cuándo conviene utilizar los algoritmos de **Kruskal** y **Prim** para construir un Árbol Cobertor Mínimo (*Minimum Spanning Tree - MST*) sobre un grafo ponderado no dirigido.

1. Resumen Comparativo

Criterio	Algoritmo de Kruskal	Algoritmo de Prim
Enfoque	Global / Basado en Aristas	Local / Basado en Vértices
Comportamiento	Ordena todas las aristas y va conectando componentes dispersos sin formar ciclos.	Hace crecer un único árbol continuo agregando el vecino más cercano paso a paso.
Estructuras Clave	Union-Find / Conjuntos Disjuntos (DSU)	Cola de Prioridad (Min-Heap) o Matriz
Complejidad	$O(E \log E)$ o $O(E \log V)$	$O(E \log V)$ con Heap / $O(V^2)$ con Matriz
Uso óptimo	Grafos Dispersos ($E \approx V$)	Grafos Densos ($E \approx V^2$)

2. Visualización del Crecimiento

Estrategia de Kruskal (Bosque de componentes disjuntos):

Kruskal selecciona las aristas con menor peso globalmente, conectando "islas" independientes:

Paso 1:

(A) (B)---(C) (D) (E)

--> Selecciona arista más barata global (B,C)

Paso 2:

(A)---(C)---(B) (D) (E)

--> Conecta (A,C)

Paso 3:

(A)---(C)---(B) (D)---(E)

--> Conecta (D,E) en otra zona

Paso 4:

(A)---(C)---(B)----- (D)---(E)

--> Une ambos componentes con (B,D)

Estrategia de Prim (Árbol en expansión continua):

Prim inicia en un vértice y expande la red de forma contigua, agregando siempre el vértice más cercano a la estructura:

Inicio:

[A] (B) (C) (D) (E)

--> Vértice inicial A

Paso 1:

[A]-----[C] (D) (E)

--> Conecta C (vecino más cercano de A)

Paso 2:

[A]----[B]-----[C] (D) (E)

--> Conecta B (vecino más cercano de {A,C})

Paso 3:

[A]----[B]-----[C]----[D] (E)

--> Conecta D (vecino más cercano de {A,B,C})

Paso 4:

[A]----[B]-----[C]----[D]----[E]

--> Conecta E (completa el MST)

3. Criterios de Selección Técnicos

- Según la Densidad del Grafo:

- **Grafos Dispersos** ($E \approx V$): Utilizar **Kruskal**. Al haber pocas aristas, ordenarlas toma poco tiempo.
- **Grafos Densos** ($E \approx V^2$): Utilizar **Prim**. Evita procesar u ordenar un volumen masivo de aristas (E).

- Según la Estructura de Datos de Entrada:

- Si la entrada es una **lista plana de aristas** (origen, destino, peso), Kruskal es más directo.
- Si la entrada está dada por una **matriz de adyacencia** o **listas de adyacencia**, Prim se adapta de forma natural.